

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1
“Основные конструкции языка Python”

Выполнил:
Студент группы ИУ5-33Б
Якубова Д. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение основных конструкций языка Python.

Задание лабораторной работы

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A , B , C , вычисляет дискриминант и **ДЕЙСТВИТЕЛЬНЫЕ** корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A , B , C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A , B , C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования (кроме C++).

Листинг кода:

```
lab_1.1.py
import sys
import math
from typing import List, Union
from dataclasses import dataclass

@dataclass
class NoRoots:
    """Нет действительных корней"""
    pass
```

```

@dataclass
class OneRoot:
    """Один действительный корень"""
    root: float

@dataclass
class TwoRoots:
    """Два действительных корня"""
    root1: float
    root2: float

@dataclass
class ThreeRoots:
    """Три действительных корня"""
    root1: float
    root2: float
    root3: float

@dataclass
class FourRoots:
    """Четыре действительных корня"""
    root1: float
    root2: float
    root3: float
    root4: float

@dataclass
class Error:
    """Ошибка в уравнении"""
    message: str

class BiquadraticEquation:
    """
    Класс для решения биквадратного уравнения вида:  $A \cdot x^4 + B \cdot x^2 + C = 0$ 
    """

    def __init__(self, a: float, b: float, c: float):
        """
        Инициализация коэффициентов биквадратного уравнения

        Args:
            a (float): коэффициент A
            b (float): коэффициент B
            c (float): коэффициент C
        """
        self.a = a
        self.b = b
        self.c = c

    def validate(self) -> Union[None, Error]:

```

```

"""
Проверка корректности уравнения

Returns:
    None если уравнение корректно, иначе Error
"""
if self.a == 0:
    return Error("Коэффициент А не может быть равен нулю для
биквадратного уравнения")
return None

def solve(self) -> Union[NoRoots, OneRoot, TwoRoots, ThreeRoots, FourRoots,
Error]:
    """
    Решение биквадратного уравнения

    Returns:
        Результат решения уравнения
    """
    # Проверяем корректность уравнения
    validation_error = self.validate()
    if validation_error:
        return validation_error

    # Решаем квадратное уравнение относительно  $y = x^2$ 
    D = self.b * self.b - 4 * self.a * self.c

    if D < 0.0:
        return NoRoots()
    elif D == 0.0:
        y = -self.b / (2.0 * self.a)
        return self._process_single_y(y)
    else:
        sqD = math.sqrt(D)
        y1 = (-self.b + sqD) / (2.0 * self.a)
        y2 = (-self.b - sqD) / (2.0 * self.a)
        return self._process_two_ys(y1, y2)

def _process_single_y(self, y: float) -> Union[NoRoots, OneRoot, TwoRoots]:
    """
    Обработка случая с одним значением y

    Args:
        y (float): значение  $y = x^2$ 

    Returns:
        Результат решения для одного y
    """
    if y < 0:
        return NoRoots()
    elif y == 0:

```

```

        return OneRoot(0.0)
    else:
        root = math.sqrt(y)
        return TwoRoots(root, -root)

def _process_two_ys(self, y1: float, y2: float) -> Union[NoRoots, OneRoot,
TwoRoots, ThreeRoots, FourRoots]:
    """
    Обработка случая с двумя значениями y

    Args:
        y1 (float): первое значение  $y = x^2$ 
        y2 (float): второе значение  $y = x^2$ 

    Returns:
        Результат решения для двух y
    """
    roots = []

    # Обработываем y1
    if y1 > 0:
        root1 = math.sqrt(y1)
        root2 = -math.sqrt(y1)
        roots.extend([root1, root2])
    elif y1 == 0:
        roots.append(0.0)

    # Обработываем y2
    if y2 > 0:
        root3 = math.sqrt(y2)
        root4 = -math.sqrt(y2)
        # Проверяем на дубликаты (если  $y1 = y2$ )
        if not (math.isclose(y1, y2) and y1 > 0):
            roots.extend([root3, root4])
    elif y2 == 0 and 0.0 not in roots:
        roots.append(0.0)

    # Убираем дубликаты и сортируем
    unique_roots = self._remove_duplicates(roots)
    unique_roots.sort()

    return self._create_result(unique_roots)

def _remove_duplicates(self, roots: List[float]) -> List[float]:
    """
    Удаление дубликатов корней с учетом погрешности вычислений

    Args:
        roots (List[float]): список корней

    Returns:

```

```

        List[float]: список уникальных корней
    """
    unique_roots = []
    for root in roots:
        if not any(math.isclose(root, ur) for ur in unique_roots):
            unique_roots.append(root)
    return unique_roots

    def _create_result(self, roots: List[float]) -> Union[NoRoots, OneRoot,
TwoRoots, ThreeRoots, FourRoots]:
        """
        Создание результата на основе списка корней

        Args:
            roots (List[float]): список корней

        Returns:
            Соответствующий объект результата
        """
        count = len(roots)
        if count == 0:
            return NoRoots()
        elif count == 1:
            return OneRoot(roots[0])
        elif count == 2:
            return TwoRoots(roots[0], roots[1])
        elif count == 3:
            return ThreeRoots(roots[0], roots[1], roots[2])
        else:
            return FourRoots(roots[0], roots[1], roots[2], roots[3])

class CoefficientReader:
    """
    Класс для чтения коэффициентов из командной строки или с клавиатуры
    """

    @staticmethod
    def get_coef(index: int, prompt: str) -> float:
        """
        Чтение коэффициента из командной строки или с клавиатуры

        Args:
            index (int): Номер параметра в командной строке
            prompt (str): Приглашение для ввода коэффициента

        Returns:
            float: Коэффициент биквадратного уравнения
        """
        while True:
            try:

```

```

        # Пробуем прочитать коэффициент из командной строки
        if len(sys.argv) > index:
            coef_str = sys.argv[index]
            coef = float(coef_str)
            return coef
        else:
            # Вводим с клавиатуры
            print(prompt)
            coef_str = input()
            coef = float(coef_str)
            return coef
    except (ValueError, IndexError):
        # Если коэффициент задан некорректно, игнорируем и запрашиваем
        заново

        if len(sys.argv) > index:
            print(f"Некорректное значение коэффициента в командной
строке. {prompt}")
            continue

class ResultPrinter:
    """
    Класс для вывода результатов решения уравнения
    """

    @staticmethod
    def print_result(result: Union[NoRoots, OneRoot, TwoRoots, ThreeRoots,
FourRoots, Error]):
        """
        Печать результата решения уравнения

        Args:
            result: Результат решения уравнения
        """
        if isinstance(result, NoRoots):
            print('Нет действительных корней')
        elif isinstance(result, OneRoot):
            print(f'Один корень: {result.root}')
        elif isinstance(result, TwoRoots):
            print(f'Два корня: {result.root1} и {result.root2}')
        elif isinstance(result, ThreeRoots):
            print(f'Три корня: {result.root1}, {result.root2}, {result.root3}')
        elif isinstance(result, FourRoots):
            print(f'Четыре корня: {result.root1}, {result.root2}, {result.root3},
{result.root4}')
        elif isinstance(result, Error):
            print(f'Ошибка: {result.message}')

def main():
    """

```

Основная функция

```
"""  
  
print("Решение биквадратного уравнения вида:  $A*x^4 + B*x^2 + C = 0$ ")  
  
# Чтение коэффициентов  
a = CoefficientReader.get_coef(1, 'Введите коэффициент A:')  
b = CoefficientReader.get_coef(2, 'Введите коэффициент B:')  
c = CoefficientReader.get_coef(3, 'Введите коэффициент C:')  
  
# Создание и решение уравнения  
equation = BiquadraticEquation(a, b, c)  
result = equation.solve()  
  
# Вывод результата  
ResultPrinter.print_result(result)  
  
if __name__ == "__main__":  
    main()
```

Скриншот работы программы

```
● uawei_d@DESKTOP-2IUV5GA:~/Yakubova-PCPL-2025$ python3 lab_1.1.py  
Решение биквадратного уравнения вида:  $A*x^4 + B*x^2 + C = 0$   
Введите коэффициент A:  
1  
Введите коэффициент B:  
1  
Введите коэффициент C:  
-2  
Два корня: -1.0 и 1.0  
○ uawei_d@DESKTOP-2IUV5GA:~/Yakubova-PCPL-2025$
```